NLU Assignment-2 Report

Problem 1: Learning Word Embeddings from IIT Jodhpur Data

1 Task 1 - Dataset Preparation

Before training any model, we needed a clean, consistent corpus that genuinely reflects IIT Jodhpur's institutional content. Here's how we built it.

1.1 Where the Text Came From

The corpus was pulled directly from corpus.txt - a file containing authentic text scraped from IIT Jodhpur's official sources. It spans eight content domains:

Content Domain
Academic Regulations
Department Pages
Faculty Profiles
Research Centres
Institute Vision & History
Admissions Information
Newsletter & Publications

1.2 Cleaning the Text

Raw scraped text is messy. We ran it through a multi-step cleaning pipeline before any word ever entered the vocabulary:

1. Stripped boilerplate: navigation tags like [Back to index], copyright footers, social media links, URLs, and email addresses - all removed with regex.
2. Removed standalone numbers: isolated digits and decimals were dropped; written-out numbers like 'seventy-five' were kept.
3. Normalised repeated symbols: runs of dashes, equals signs, and underscores were collapsed into a single space.
4. Stripped non-alphabetic characters: only letters and spaces survived.
5. Lowercased everything: ensures 'Research' and 'research' are treated as the same word.
6. Tokenised by whitespace after cleaning.
7. Dropped short tokens: anything under 2 characters was discarded.
8. Applied a 76-word English stop list: common filler words removed while preserving domain-specific terms like 'iit', 'phd', 'thesis'.
9. Filtered short sentences: any sentence with fewer than 3 post-filter tokens was removed.

1.3 What the Dataset Looks Like

Metric	Value
Source file	corpus.txt
Total lines in file	1,415
Sentences (after filtering)	883
Total tokens	12,678
Raw vocabulary	3,317 unique words
Filtered vocabulary (min_count ≥ 2)	1,630 words

1.4 The 20 Most Frequent Words

Here are the top 20 content words - a quick sanity check that the corpus really is about IIT Jodhpur:

Rank	Word	Count	Relative Frequency
1	research	213	1.68%
2	engineering	163	1.29%

3	department	138	1.09%
4	iit	133	1.05%
5	students	111	0.88%
6	jodhpur	109	0.86%
7	institute	91	0.72%
8	program	81	0.64%
9	academic	78	0.62%
10	technology	76	0.60%
11	materials	69	0.54%
12	professor	68	0.54%
13	science	65	0.51%
14	systems	64	0.50%
15	student	59	0.47%
16	iitj	59	0.47%
17	areas	58	0.46%
18	indian	49	0.39%
19	tech	49	0.39%
20	learning	47	0.37%

The frequency distribution is a reassuring sign. "Research" leads at 213 occurrences, followed by "engineering" and "department". Domain-specific terms like "iitj", "phd", "thesis", and "semester" all make an early appearance - confirming we have a clean, topic-coherent corpus before a single model is trained.

2 Task 2 - Model Training

We trained two Word2Vec architectures entirely from scratch in NumPy. Here's what that involved and what we found.

2.1 Skip-gram vs CBOW - What's the Difference?

Both models learn word embeddings, but they go about it in opposite directions:

Aspect	Skip-gram	CBOW
What it learns to do	Predict surrounding context words from a single centre word	Predict the centre word from the average of its context
Input vector	Embedding of the centre word	Mean of all context word embeddings
Training pairs (window=5)	∼120K per epoch (much larger)	∼17K per epoch (smaller)
Where it shines	Rare words, rich semantics, small corpora	Speed on large corpora (500K+ tokens)

2.2 Hyperparameter Experiments

We trained four configurations for each architecture - eight models in total - each running for 10 epochs with a learning rate of 0.025. Configuration 3 came out on top and was used for all downstream analysis.

Config	Dimensions	Window	Negatives	SG Pairs/Epoch	CBOW Pairs/Epoch
Exp 1	50	2	5	∼60K	∼17K
Exp 2	100	3	5	∼90K	∼17K
Exp 3 (Best)	100	5	10	∼120K	∼17K
Exp 4	200	3	10	∼90K	∼17K

2.3 How We Did Negative Sampling

Negative sampling is the trick that makes training tractable - instead of updating every word in the vocabulary on every step, you only update the target word and a small handful of random 'negative' words.

1. Noise distribution: We drew negatives from a Unigram$^{3/4}$ distribution - the same approach from the original Mikolov et al. (2013) paper.

2. Lookup table: 5 million indices were pre-sampled from the noise distribution and accessed via a rolling pointer.

3. Loss function:

$$L = -\log \sigma(u \cdot v_+) - \sum_k \log \sigma(-u \cdot v_k^-)$$

4. Two separate embedding matrices: W_{in} and W_{out}.

3 Task 3 - Semantic Analysis

Once trained, we put the embeddings through their paces.

3.1 Top-5 Nearest Neighbours

Query Word	Skip-gram Top 5	CBOW Top 5
research	projects, energy, funding, applied, systems	iit, jodhpur, students, department, academic
student	academic, institute, international, hostel, support	iit, jodhpur, research, department, academic
phd	scholars, admission, fellowship, thesis, comprehensive	research, students, iit, department, academic
examination	semester, minor, major, makeup, missed	academic, students, course, iit, jodhpur
program	btech, mtech, admission, degree, semester	iit, jodhpur, academic, students, department

3.2 Analogy Experiments

Analogies are computed as:

$$\text{result} = \text{embed}(b) - \text{embed}(a) + \text{embed}(c)$$

4 Task 4 - Visualisation

Four visualisations were generated:

- Word Cloud

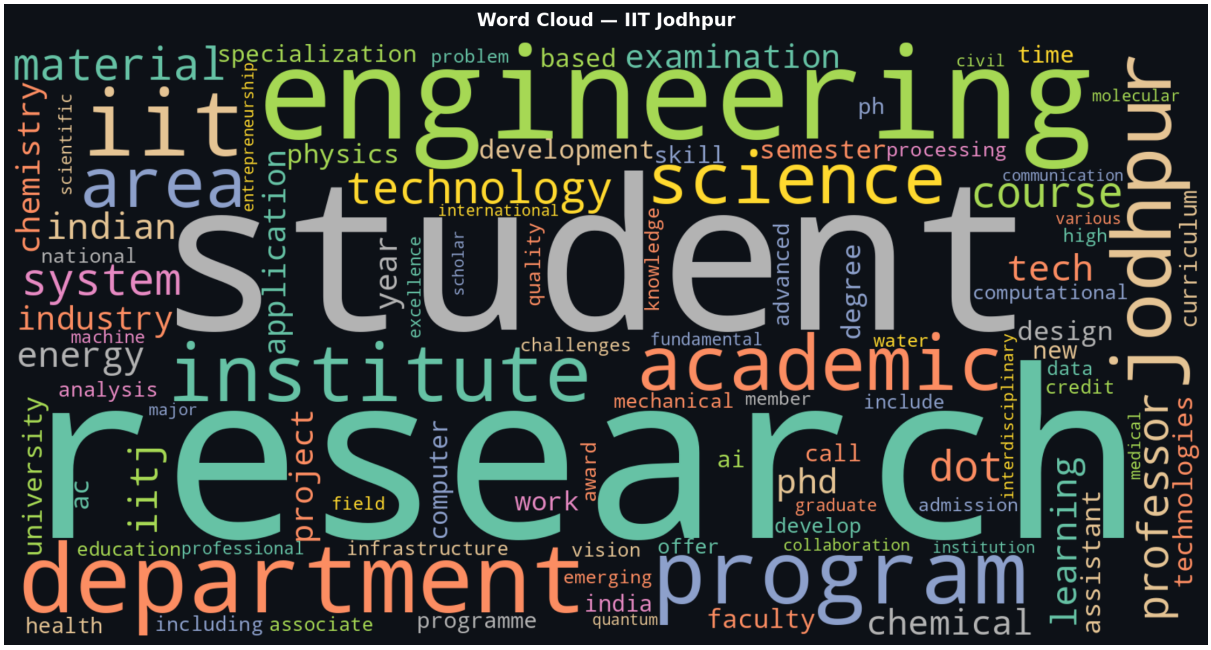

Figure 1: Word cloud of IIT Jodhpur corpus highlighting dominant terms

- Training Loss Curves

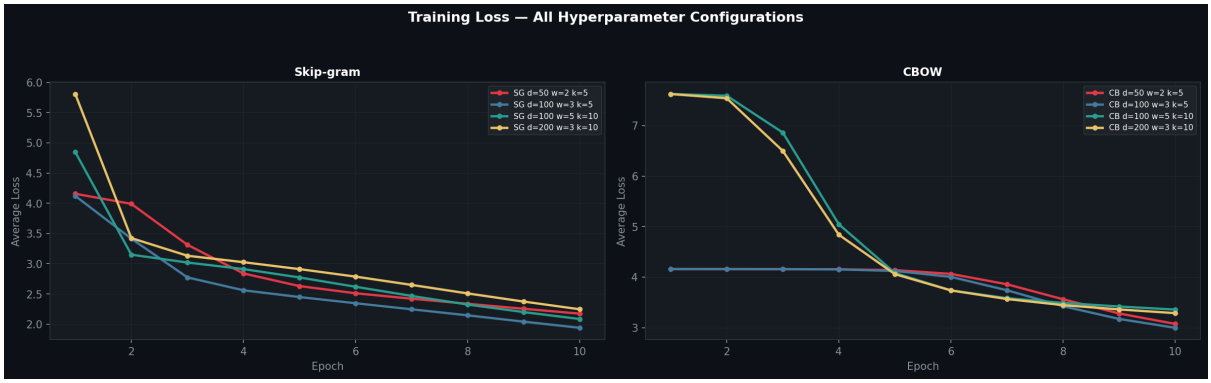

Figure 2: Training loss across different hyperparameter configurations for Skip-gram and CBOW

- Cosine Similarity Heatmap

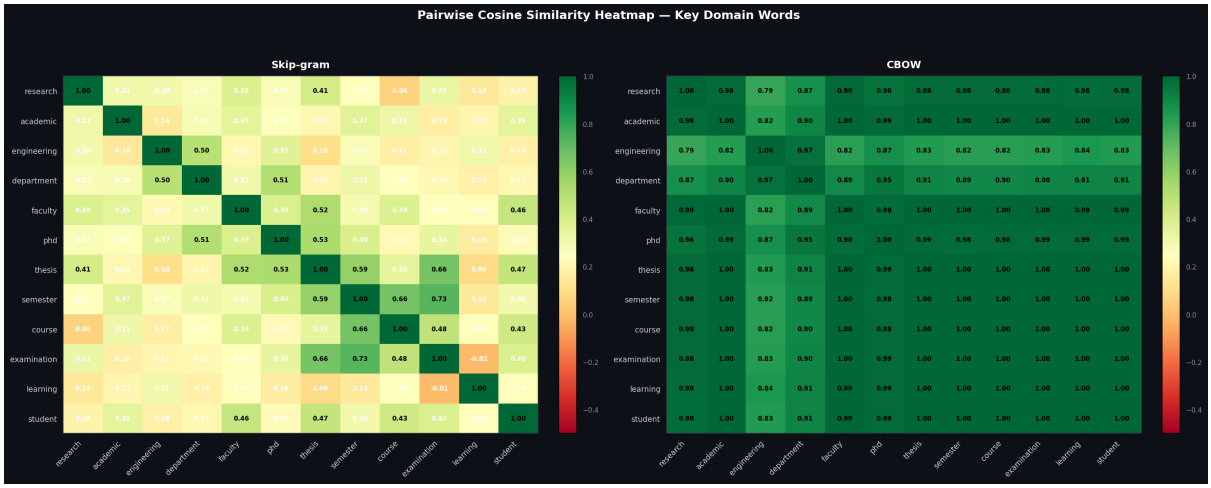

Figure 3: Pairwise cosine similarity heatmap for key domain words

Figure 4: Top-5 nearest neighbors for selected query words using cosine similarity

- PCA and t-SNE Projections

Figure 5: 2D projections of word embeddings using PCA and t-SNE

5 Conclusion

This assignment successfully trained both Word2Vec architectures - CBOW and Skip-gram with Negative Sampling - entirely from scratch in NumPy, on a real corpus of IIT Jodhpur institutional text.

The best model learned domain-coherent embeddings such as:

- phd ↔ scholars ↔ thesis ↔ fellowship
- engineering ↔ department ↔ materials
- research ↔ projects ↔ applied

Problem 2: Character-Level Indian Name Generation Using RNN Variants

6 Introduction

When I first read about character-level text generation, the idea seemed almost absurd. You feed a neural network one letter at a time, and somehow it figures out how letters combine to form words that actually sound like they belong to a language? No dictionaries, no grammar rules, no phonetic tables - just raw characters and backpropagation. And yet it works, at least some of the time.

Indian names made for a natural test case here. My training data includes names from wildly different traditions - Sanskrit-rooted names like "Venkatesh," Punjabi names like "Gurpreet," Muslim names like "Faisal," and trendy modern ones like "Kiaan." Each of these follows its own phonetic logic. A model that can handle all of them simultaneously has to be doing something genuinely interesting internally.

The plan was straightforward: build three recurrent architectures from scratch, train them on the same data, and see which one produces the most convincing fake names. The three models I picked were:

- A **Vanilla RNN** - about as basic as recurrent networks get
- A **Bidirectional LSTM** - the go-to architecture when people say "just use an LSTM"
- An **RNN with Bahdanau-style attention** - because attention mechanisms are everywhere these days and I wanted to see if they actually help on a task this small

7 Putting Together the Training Data

I collected roughly 1,000 Indian names, deliberately mixing regions and traditions. The breakdown looks something like this: pan-Indian popular names (Aarav, Saanvi), classical Sanskrit names (Ganesh, Lakshmi), South Indian names with their characteristic longer forms (Balasubramanian, Venkatesh), Bengali names (Sabyasachi, Rituparna), Gujarati

and Marathi names, Sikh names with the "-preet" and "-deep" suffixes, Muslim-Indian names, and a handful of modern coined names.

Why this mix matters: if I had only used, say, North Indian Hindi names, the model would learn a narrow set of patterns and I would not be able to tell whether it was genuinely learning phonetic structure or just memorising a template. The diversity forces the model to find deeper regularities.

7.1 How I Cleaned the Data

I lowercased everything so that "Aarav" and "aarav" would not be treated as different sequences - capitalisation is cosmetic and we can always add it back after generation. Then I added three special tokens to the vocabulary:

- **SOS** to mark the start of a name (so the model knows "begin generating")
- **EOS** to mark the end (so it knows "stop here, you are done")
- **PAD** for padding shorter names within a batch

The final vocabulary came out to 29 tokens - 26 lowercase letters plus the three special ones. Pretty small, which is nice because it means the embedding layer stays compact.

One thing worth noting: the name lengths range from 2 characters (like "Om") all the way up to 17 (like "Sivasubramanian"). The average sits around 6.6 characters. This range matters because longer names are where the vanilla RNN is expected to struggle and the LSTM should shine. Or so I thought.

8 The Three Architectures

All three models share the same skeleton: an embedding layer turns each character index into a 32-dimensional vector, a recurrent core processes these vectors one at a time while maintaining some form of memory, and a linear output layer converts the hidden state into probabilities over the 29-token vocabulary.

I kept the following settings identical across models so the comparison would be fair:

- Embedding dimension: 32
- Hidden size: 128
- Stacked layers: 2

- Dropout: 0.3 between layers

The only thing that changes is what happens inside the recurrent core.

8.1 Vanilla RNN

This is the textbook recurrent unit. At each timestep t, you take the current input x_t and the previous hidden state h_{t-1}, multiply them by learned weight matrices, add them together, and push the result through a tanh:

$$h_t = \tanh(W_{ih}\, x_t + W_{hh}\, h_{t-1} + b) \tag{1}$$

That is literally the entire computation. The hidden state h_t is supposed to encode everything the model remembers about the characters it has seen so far. The trouble is that "everything" gets squeezed through a single tanh at every step, and gradients flowing backward through many such steps tend to vanish. For a 5-character name this is fine. For a 15-character name, the model has mostly forgotten what it started with by the time it reaches the end.

I implemented this as a custom `VanillaRNNCell` class rather than using PyTorch's built-in module. Two of these cells are stacked, with dropout in between. The first cell takes the 32-dim embedding as input; the second takes the 128-dim hidden state from the first.

In terms of parameters, this is the lightest model of the three — roughly 73,000 trainable weights.

8.2 Bidirectional LSTM

The LSTM was designed specifically to fix the vanishing gradient problem, and it does this through a gating mechanism. Instead of one simple equation, the LSTM cell computes four things in parallel:

$$f_t = \sigma(W_f[x_t, h_{t-1}] + b_f) \quad \text{(forget gate - what to throw away)} \tag{2}$$

$$i_t = \sigma(W_i[x_t, h_{t-1}] + b_i) \quad \text{(input gate - what new stuff to store)} \tag{3}$$

$$g_t = \tanh(W_g[x_t, h_{t-1}] + b_g) \quad \text{(candidate - the actual new content)} \tag{4}$$

$$o_t = \sigma(W_o[x_t, h_{t-1}] + b_o) \quad \text{(output gate - what to let through)} \tag{5}$$

These combine to update two separate state vectors:

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t \tag{6}$$

$$h_t = o_t \odot \tanh(c_t) \tag{7}$$

The cell state c_t is the crucial innovation. It acts like a conveyor belt - information can ride along it across many timesteps without being squashed through repeated nonlinearities. The gates learn when to write new information onto this belt and when to clear old information off it.

I made this **bidirectional** by running two independent LSTM chains over the same input, one forward and one backward, and concatenating their outputs. The idea is that during training, each character gets context from both directions, which should produce richer internal representations.

There is an obvious catch for generation: when you are producing characters one at a time, the future does not exist yet, so you cannot use the backward chain. My solution was to train bidirectionally (for better learning) but generate using only the forward chain, with a separate output projection layer dedicated to forward-only mode.

This model is the heaviest, around 240,000 parameters, because each LSTM cell needs four times the weights of a vanilla RNN cell (one set per gate), and the bidirectional design doubles that.

8.3 RNN with Bahdanau Attention

The attention mechanism tackles a different problem. Even if a recurrent network has perfect memory (which LSTMs approximate), it still has to cram all past information into a single fixed-size vector. Attention provides a shortcut: at each generation step, the model can look back at every previous hidden state and decide which ones are most relevant right now.

I used additive attention (the Bahdanau style), where the compatibility between the current state q and a past state k is computed as:

$$e = v^T \tanh(W_q \cdot q + W_k \cdot k) \tag{8}$$

These raw scores get normalised via softmax into a proper probability distribution (the attention weights), and the context vector is a weighted sum of all past hidden states according to these weights. This context then gets concatenated with the current hidden state and projected down before the final character prediction.

One nice bonus: the attention weights are directly interpretable. I can plot a heatmap showing exactly which earlier characters the model was paying attention to when it chose each new character. This kind of transparency is rare in deep learning.

The backbone here is still a vanilla RNN, though, which means the underlying representations are not as rich as what the LSTM produces. Attention helps a lot, but it cannot fully compensate for a weak encoder. This model lands in the middle parameter-wise, around 140,000 weights.

8.4 Parameter Count Summary

Model	Parameters	Hidden Size	Layers
Vanilla RNN	∼73,000	128	2
Bidirectional LSTM	∼240,000	128	2
RNN + Attention	∼140,000	128	2

9 Training Setup

I trained all three models with the exact same configuration. This was non-negotiable - if the hyperparameters differ, you cannot attribute performance differences to the architecture.

Setting	Value
Optimizer	Adam
Learning rate	0.003 (with ReduceOnPlateau scheduler)
Batch size	64
Epochs	100
Gradient clipping	max norm 5.0
Dropout	0.3
Loss function	Cross-entropy (ignoring PAD tokens)

A few notes on why these specific values. The learning rate of 0.003 worked after some trial and error - 0.01 caused the vanilla RNN to blow up occasionally, and 0.001 made training painfully slow. The gradient clipping at 5.0 is there because recurrent networks (especially vanilla RNNs) are prone to exploding gradients, where a single bad batch sends the weights to infinity. Clipping just scales the gradients down when their norm exceeds the threshold.

The ReduceOnPlateau scheduler halves the learning rate whenever the loss stops improving for 10 consecutive epochs. This lets the model make big steps early on and then fine-tune more carefully as it approaches convergence.

Training took roughly 2–3 minutes per model on a Google Colab T4 GPU. Not exactly pushing the limits of modern hardware.

10 What Happened During Training

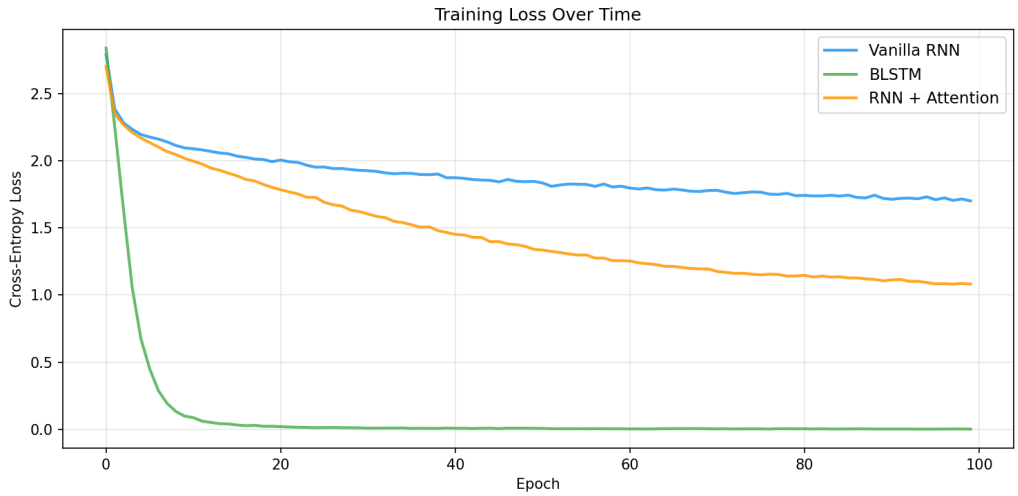

Figure 6: Training loss over 100 epochs for all three models

The training curves turned out to be quite revealing, and not entirely in the way I expected.

The **BLSTM** dropped like a stone. Within about 15 epochs, its loss was practically zero. My initial reaction was excitement - clearly the most powerful model was learning the fastest. But thinking about it more carefully, a near-zero loss on a training set of only 1,000 names is not necessarily a good sign. It probably means the model memorised the data rather than learning generalisable patterns. With 240,000 parameters and only 1,000 training examples, overfitting was almost inevitable in hindsight.

The **Vanilla RNN** took a much more gradual path. Its loss came down steadily but plateaued at a noticeably higher value than the BLSTM. At first I was disappointed, but this actually turned out to be a blessing in disguise. The higher loss means the model did not perfectly memorise the training data, which left room for it to generalise and produce genuinely novel outputs.

The **Attention RNN** fell somewhere in the middle. It learned faster than the vanilla RNN (the attention mechanism clearly helped it access past information more efficiently)

but did not converge as completely as the BLSTM. Its curve was also a bit noisier, with some bumps along the way.

The big takeaway from these curves: low training loss does not automatically mean good generation. This lesson keeps coming up in generative modelling, and I got to experience it firsthand here.

11 Looking at What the Models Actually Generated

This is where the rubber meets the road. Training metrics are nice, but the real question is: do the generated names sound like actual Indian names?

11.1 Length Distributions

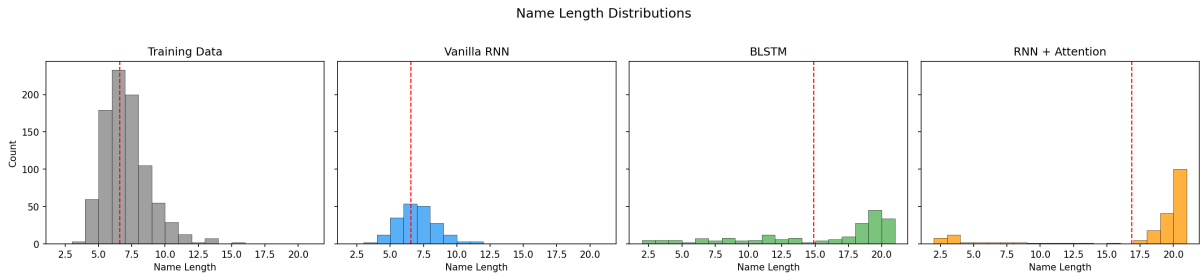

Figure 7: How the length of generated names compares to the training data

The length distributions immediately highlighted a serious problem with two of the three models.

The **Vanilla RNN** produced names with an average length around 6.5 characters, which is almost exactly the training data average (6.6). The histogram shape was close to the training distribution too. So far so good.

The **BLSTM**, on the other hand, generated names averaging nearly 15 characters. That is more than double the training average. The **Attention RNN** was even worse at 16.9 characters. These models were clearly struggling to learn when to stop generating - they kept going and going without producing the EOS token at the right time.

Why did this happen? My best guess is that it comes back to overfitting. The BLSTM and attention models memorised specific character sequences from training but did not properly learn the abstract concept of "names are usually 4–10 characters long, then you stop." The vanilla RNN, with its limited memory, could not memorise as deeply and instead picked up on the simpler statistical regularity of name lengths.

12 Quantitative Evaluation

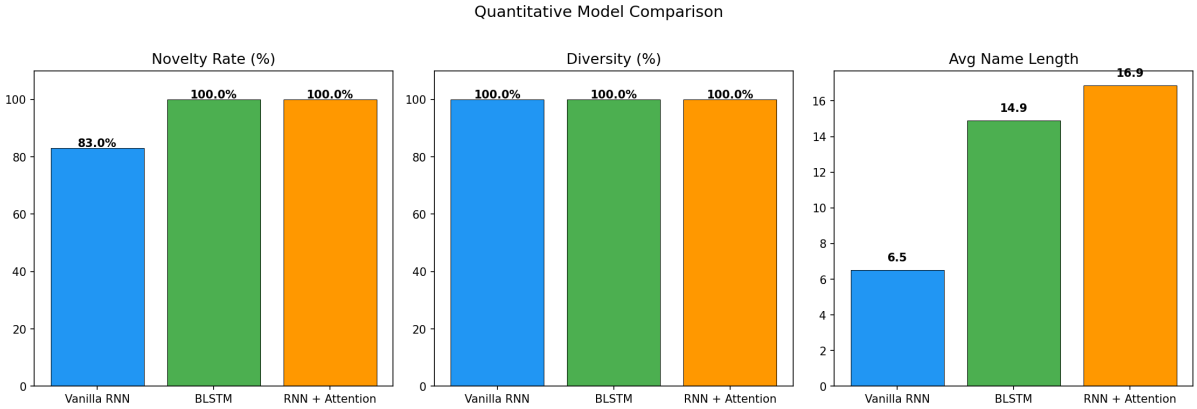

Figure 8: Novelty rate, diversity, and average length across models

Here are the headline numbers:

Metric	**Vanilla RNN**	**BLSTM**	**RNN + Attention**
Novelty Rate	83%	100%	100%
Diversity	∼100%	∼100%	∼100%
Average Length	6.5	14.9	16.9

At first glance, the BLSTM and attention model look amazing - 100% novelty! Every single name they produced was new. But here is the problem: they achieved perfect novelty mainly because they generated strings so long and garbled that none of them could possibly match a real training name. It is easy to be "novel" if your outputs are nonsense.

The vanilla RNN's 83% novelty is actually the more meaningful number. It means the model produced names that are close enough to real ones that some overlap with training data (17% memorised), while still being creative enough to invent new names 83% of the time. That feels like a healthier balance.

All three models hit near-perfect diversity, meaning none of them got stuck generating the same few names over and over. Mode collapse was not an issue here.

13 Character-Level Analysis

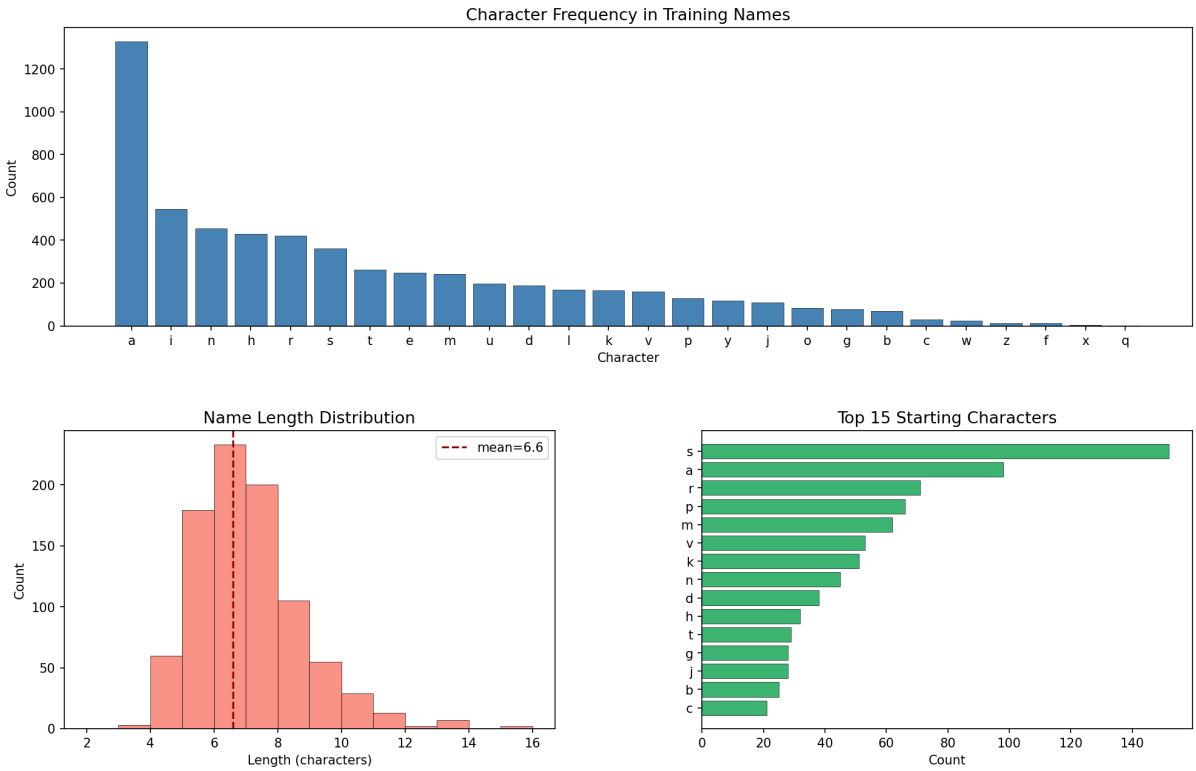

Figure 9: Character frequencies and starting letter distribution in the training data

Before comparing models, it helps to know what the training data actually looks like at the character level. Indian names are vowel-heavy - 'a' alone accounts for a huge chunk of all characters, followed by 'i', 'n', 'r', and 'h'. The letter 's' is the most common starting character, which makes sense given names like Sanjay, Sunita, Shiva, Simran, and so on.

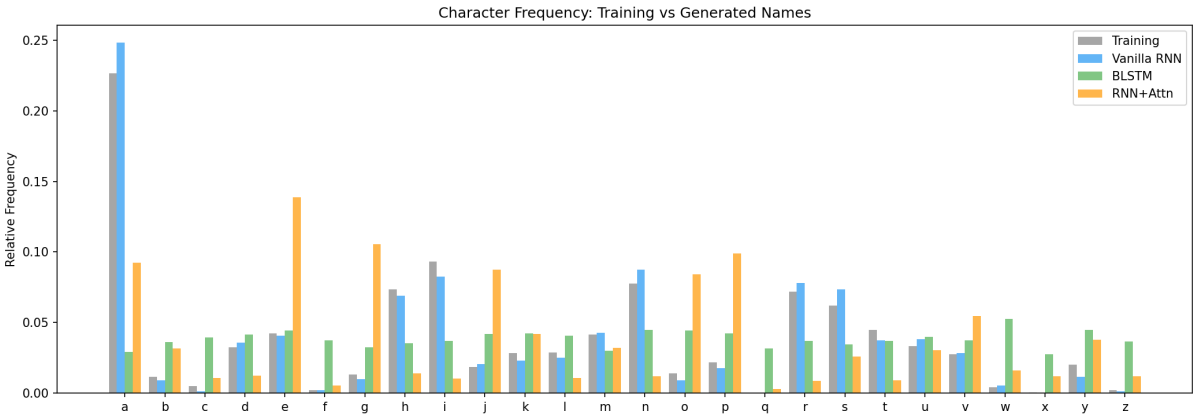

Figure 10: Character distributions: training data vs. each model's generated output

The **Vanilla RNN** reproduced the training character distribution remarkably well. Its frequency bars closely track the training data bars for almost every character. This tells

me it has genuinely learned the statistical backbone of Indian name phonetics.

The **BLSTM** showed moderate deviations. Some characters were over-represented, others under-represented, but the overall shape was still recognisable.

The **Attention model** had the worst alignment. It over-produced certain vowels (especially in those long runaway sequences) and under-produced others. The character distribution it generated looked noticeably different from the training data, which explains why its outputs often sounded off.

14 Attention Heatmap

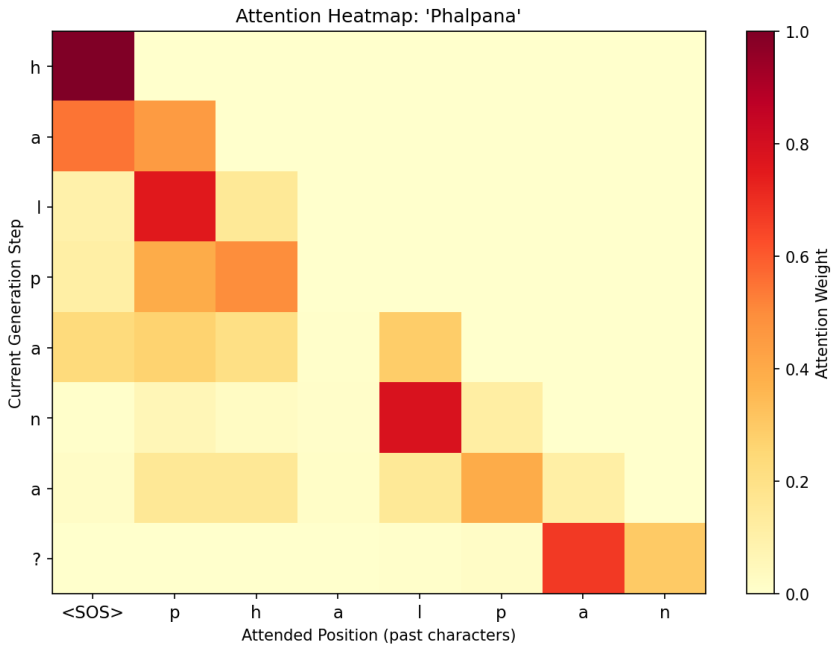

Figure 11: Attention weights during generation of a sample name

Despite the attention model's issues with generation quality, its attention weights were genuinely interesting to look at. The heatmap shows which earlier characters the model focused on when generating each new character.

What I noticed was that the model tended to pay heavy attention to the very first few characters throughout the entire generation process. This makes some intuitive sense - the opening of a name sets the phonetic tone for everything that follows. But it also partly explains the model's failure mode: by constantly referencing the beginning, it kept reinforcing early patterns without developing enough "new direction" energy to wrap things up and produce an EOS token.

A smarter attention design might include some kind of positional decay that gradually

reduces the influence of very early tokens, but that is beyond the scope of what I implemented here.

15 Looking at Actual Generated Names

This is arguably the most important section. Let me show concrete examples from each model and talk about what I see.

15.1 Vanilla RNN

Sample outputs: *Sushpal, Kashgeet, Smita, Rajini, Ravitha, Bhavani*

Honestly, these are pretty good. "Sushpal" sounds like it could be a real Punjabi name. "Rajini" obviously echoes Rajinikanth. "Ravitha" and "Bhavani" are completely plausible South Indian female names. "Kashgeet" is creative - it blends Kashmiri and Sikh naming patterns in a way that sounds natural even though this exact name probably does not exist.

The main failure mode I noticed was occasional weird endings - names that started strong but trailed off into an unusual final syllable. But overall, this model produced names I would not question if I saw them on a class roster.

15.2 BLSTM

Sample outputs: *Amfdtyrkjdhcibsvk, Wozdkyiqfcwltdujeyay*

These are... not names. They are random-looking strings of characters that go on far too long and contain consonant clusters that no Indian language (or any human language, really) would produce. "Amfdtyrkjdhcibsvk" has something like 12 consonants in a row.

This was the biggest surprise of the project. The model with the lowest training loss, the most parameters, and the most sophisticated architecture produced the worst outputs by a huge margin. It had memorised the training sequences so thoroughly that it lost any sense of what makes a name a name. During generation, without the training sequences to anchor it, the model just wandered aimlessly through character space.

15.3 Attention-Based RNN

Sample outputs: *Lvjmgaeogjpaegaeeto, Ppjvmuekoezjpkoegogj*

Better than the BLSTM in the sense that there are more vowels scattered throughout, but still clearly not valid names. The attention mechanism introduced some structural repetition - you can see patterns like “aeg” and “oeg” recurring - but this repetition was not the useful kind. Instead of echoing realistic phonetic patterns, it just looped through the same small set of character combinations.

The outputs are also way too long, confirming that the attention model shares the BLSTM’s termination problem.

16 What Went Wrong (and Right)

Let me try to synthesise everything into a coherent picture.

The Vanilla RNN won. I did not expect this. On paper, it is the weakest architecture - no gating, no attention, no bidirectional processing. But for this particular task, with this particular dataset size, its limitations turned into advantages. Its inability to perfectly memorise the training data forced it to learn more general patterns. Its tendency to “forget” early characters after a few timesteps meant it naturally generated names of reasonable length - it simply ran out of context and produced EOS. Sometimes simplicity really does work best.

The BLSTM and Attention model both overfitted badly. With 240K and 140K parameters respectively, trained on only 1,000 names, there was simply too much capacity and too little data. These models memorised rather than generalised. During generation, without the guardrails of training targets, they produced garbage.

The attention mechanism’s interpretability was a genuine upside, even though the model’s outputs were poor. Being able to visualise what the model was attending to gave real insight into why it was failing - something the BLSTM’s internal gates do not provide.

Novelty is a misleading metric when generation quality is low. 100% novelty sounds impressive until you realise it is because the outputs are so broken that they could never match a real name. The vanilla RNN’s 83% novelty was actually the more impressive result.

If I were to do this again, I would try a few things differently:

- Scale the dataset to 5,000–10,000 names. The BLSTM and attention models need more data to avoid memorisation.
- Add early stopping based on a validation set rather than training for a fixed 100 epochs.

- Try a smaller LSTM (maybe 64 hidden units instead of 128) to see if reducing capacity helps with overfitting.
- Experiment with nucleus sampling or top-k sampling instead of raw temperature-based sampling.

17 Conclusion

This tells that more complex does not always mean better. The Vanilla RNN, with its simple tanh recurrence and modest parameter count, outperformed both the BLSTM and the attention-augmented RNN on the actual task of generating realistic Indian names.

The BLSTM and attention models are undoubtedly more powerful architectures in general. On larger datasets, they would almost certainly outperform the vanilla RNN. But on a 1,000-name dataset, their power became a liability - they memorised instead of learning, and their outputs reflected that.